

Module 4

Robot Framework — Keyword-Driven Test Automation

Module 4 of 4 — Selenium / QA Automation Training
Detailed Curriculum Report with Explanations, Examples & Code

3.1 Introduction to Robot Framework

Robot Framework is a generic, open-source, **keyword-driven** automation framework. Unlike unittest/PyTest where tests are Python code, Robot Framework tests are written in a simple tabular, plain-text syntax made of **keywords** (readable phrases like Click Element or Input Text), making it approachable for both technical and non-technical team members. It uses the SeleniumLibrary under the hood for web automation.

Environment Setup

Installation

```
pip install robotframework
pip install robotframework-seleniumlibrary
pip install robotframework-ride      # optional GUI editor (RIDE)

robot --version
```

Robot Framework — RIDE

RIDE (Robot Integrated Development Environment) is a standalone graphical editor for writing and organizing Robot Framework test suites without hand-editing text files. Modern practice increasingly favors writing .robot files directly in VS Code (with the Robot Framework Language Server extension) instead of RIDE.

Writing a Simple Test Case

login_tests.robot

```
*** Settings ***
Library      SeleniumLibrary

*** Variables ***
${URL}       https://www.saucedemo.com
${BROWSER}   chrome

*** Test Cases ***
Open Browser And Verify Title
    Open Browser    ${URL}    ${BROWSER}
    Title Should Be    Swag Labs
    Close Browser
```

Keyword-Driven and Data-Driven Test Cases

A **keyword** is a reusable, named block of steps (similar to a function). Robot Framework ships with hundreds of built-in keywords (from BuiltIn and SeleniumLibrary), and you can compose your own from them.

keyword_driven.robot

```

*** Settings ***
Library      SeleniumLibrary

*** Variables ***
${URL}       https://www.saucedemo.com

*** Test Cases ***
Valid Login
    Open Browser    ${URL}    chrome
    Input Text      id:user-name    standard_user
    Input Text      id:password     secret_sauce
    Click Button    id:login-button
    Page Should Contain Element    id:inventory_container
    Close Browser

*** Keywords ***
Login As
    [Arguments]    ${username}    ${password}
    Input Text      id:user-name    ${username}
    Input Text      id:password     ${password}
    Click Button    id:login-button

```

Data-driven variant using the Template keyword:

data_driven.robot

```

*** Test Cases ***
Login With Multiple Users
    [Template]    Login As
    standard_user    secret_sauce
    locked_out_user    secret_sauce
    problem_user    secret_sauce

```

Writing and Executing Test Cases (Command Line)

Running from the command line

```

robot login_tests.robot
robot --outputdir results login_tests.robot
robot --loglevel DEBUG login_tests.robot

```

Test Setup and Teardown

setup_teardown.robot

```
*** Settings ***
Library          SeleniumLibrary
Suite Setup      Open Browser      https://www.saucedemo.com    chrome
Suite Teardown   Close All Browsers
Test Setup       Go To            https://www.saucedemo.com
Test Teardown    Capture Page Screenshot

*** Test Cases ***
Valid Login
    Input Text    id:user-name    standard_user
    Input Text    id:password     secret_sauce
    Click Button  id:login-button
    Page Should Contain Element    id:inventory_container
```

Working with Browsers Using the SeleniumLibrary

browser_control.robot

```
*** Test Cases ***
Cross Browser Demo
    Open Browser      https://www.saucedemo.com    firefox
    Maximize Browser Window
    Wait Until Element Is Visible    id:user-name    timeout=10s
    Close Browser
```

3.2 Script Running Options

Robot vs. Pybot

In older Robot Framework versions, the command-line runner for Python-based tests was called pybot (with jybot for Jython). Since Robot Framework 3.1+, these were unified into a single robot command, which is what current projects use exclusively.

Running from PyCharm, Command Window, Batch File, Task Scheduler

- **PyCharm:** install the “Robot Framework Support” plugin, then right-click a .robot file → Run.
- **Command Window (cmd/PowerShell):** robot tests\login_tests.robot
- **Batch File:** wrap the command in a .bat file for one-click / scheduled execution.
- **Windows Task Scheduler:** point a scheduled task at the batch file to run the suite automatically (e.g. nightly regression).

run_tests.bat

```
@echo off
cd C:\automation\robot_tests
robot --outputdir results login_tests.robot
```

Running on SauceLabs.com Browsers

SauceLabs provides cloud-hosted real/virtual browsers. Robot Framework connects to it by pointing Open Browser at SauceLabs' remote WebDriver URL with the required capabilities.

saucelabs.robot

```
*** Test Cases ***
Run On SauceLabs
  Open Browser    https://www.saucedemo.com
  ...    remote_url=https://USERNAME:ACCESS_KEY@ondemand.saucelabs.com:443/wd/hub
  ...    desired_capabilities=browserName:chrome;platformName:Windows 11;browserVersion:latest
  Close Browser
```

Running & Scheduling from Jenkins

- Install the “Robot Framework” Jenkins plugin to visualize results as trend graphs.
- Add a Jenkins build step / pipeline stage that runs robot and publishes the output.
- Schedule via Jenkins' cron-style build triggers (e.g. H 2 * * * for a nightly run).

Jenkinsfile

```
// Jenkinsfile (declarative pipeline)
pipeline {
    agent any
    triggers { cron('H 2 * * *') }
    stages {
        stage('Run Robot Tests') {
            steps {
                sh 'robot --outputdir results tests/'
            }
        }
    }
    post {
        always {
            step([$class: 'RobotPublisher', outputPath: 'results',
                outputFileName: 'output.xml', passThreshold: 90, unstableThreshold: 70])
        }
    }
}
```

Running Single Test Cases, Multiple Suites & Advanced Run Options

Advanced CLI run options

```
robot --test "Valid Login" login_tests.robot      # a single named test case
robot tests/suite1.robot tests/suite2.robot        # multiple suites
robot --suite Login --suite Checkout tests/        # run specific suites in a directory
robot --include smoke tests/                      # run by tag
robot --exclude wip tests/                        # exclude by tag
robot --processes 4 tests/                        # parallel run (via pabot)
```

3.3 Making the Script More Readable

Procedural vs. Gherkin Style

Robot Framework test steps can be written either in a plain procedural style (a flat list of keyword calls) or in a Gherkin-like BDD style using the built-in Given / When / Then / And prefixes, which are simply stripped and treated as ordinary keyword calls, improving readability for non-technical stakeholders.

procedural_vs_gherkin.robot

```
*** Test Cases ***
# Procedural style
Valid Login Procedural
    Open Browser      https://www.saucedemo.com    chrome
    Input Text        id:user-name    standard_user
    Input Text        id:password    secret_sauce
    Click Button      id:login-button
    Page Should Contain Element    id:inventory_container

# Gherkin style
Valid Login Gherkin
    Given I am on the login page
    When I log in with valid credentials
    Then I should see the inventory page
```

Introducing User-Defined Keywords

User-defined keywords backing the Gherkin steps

```
*** Keywords ***
I am on the login page
    Open Browser      https://www.saucedemo.com    chrome

I log in with valid credentials
    Input Text        id:user-name    standard_user
    Input Text        id:password    secret_sauce
    Click Button      id:login-button

I should see the inventory page
    Page Should Contain Element    id:inventory_container
```

Breaking the Script into Keywords

Large, monolithic test cases should be decomposed into small, single-purpose keywords (Login, AddItemToCart, Checkout) that can be reused across many test cases — mirroring the Page Object Model idea from Module 2, but expressed as Robot Framework keywords instead of Python methods.

Moving Keywords to Resource Files

resources/login_keywords.robot

```
# resources/login_keywords.robot
*** Settings ***
Library    SeleniumLibrary

*** Keywords ***
Login As
    [Arguments]    ${username}    ${password}
    Input Text     id:user-name    ${username}
    Input Text     id:password    ${password}
    Click Button   id:login-button
```

login_tests.robot (imports the resource file)

```
# login_tests.robot
*** Settings ***
Resource    resources/login_keywords.robot

*** Test Cases ***
Valid Login
    Open Browser    https://www.saucedemo.com    chrome
    Login As        standard_user    secret_sauce
    Page Should Contain Element    id:inventory_container
```

Adding Setup and Teardown at Keyword Level

Keyword-level teardown

```
*** Keywords ***
Login As
    [Arguments]    ${username}    ${password}
    [Teardown]     Capture Page Screenshot
    Input Text     id:user-name    ${username}
    Input Text     id:password    ${password}
    Click Button   id:login-button
```

Overview of Page Objects in Robot Framework

Robot Framework achieves the same goal as Selenium's Page Object Model by placing each page's locators and keywords into its own resource file (as shown above), which is then imported into the relevant test suites. This keeps locators centralized and out of the test cases themselves.

Page-object-style project structure

```
resources/
  login_page.robot
  inventory_page.robot
  checkout_page.robot
tests/
  login_tests.robot
  checkout_tests.robot
```

Increasing Readability Using Gherkin

Fully Gherkin-readable test case

```
*** Test Cases ***
Successful Checkout
    Given I am logged in as "standard_user"
    And I have added "Sauce Labs Backpack" to the cart
    When I proceed to checkout
    And I complete the checkout form
    Then I should see the order confirmation message
```

Tags and Reports/Logs

Tagging a test case

```
*** Test Cases ***
Valid Login
    [Tags]    smoke    login
    ...
```

Generating reports

```
robot --include smoke --outputdir results tests/
# Generates: results/report.html, results/log.html, results/output.xml
```

Custom log messages for debugging

```
*** Test Cases ***
Valid Login
    Log    Starting the valid-login test    level=INFO
    ...
```

Module 4 Summary: You learned Robot Framework's keyword-driven approach as an alternative to Python-code-based automation — installation, writing and running test cases, data-driven templates, execution from PyCharm/CLI/batch files/SauceLabs/Jenkins, and how to structure readable, maintainable suites using user-defined keywords, resource files, Gherkin-style steps, tags, and HTML reports/logs. Together with Modules 1–3, you now have three complementary automation approaches: raw Selenium + PyTest/POM, Python BDD for REST APIs, and Robot Framework's keyword-driven style.